Заглавная страница
Содержание
Избранные статьи
Случайная статья
Текущие события
Пожертвовать

Участие
Сообщить об ошибке
Как править статьи
Сообщество
Форум
Справка
Свежие правки
Новые страницы
Служебные страницы

Инструменты
Ссылки сюда
Связанные правки
Постоянная ссылка
Сведения о странице
Цитировать страницу
Получить короткий URL
Скачать QR-код
Развернуть всё

Печать/экспорт
Скачать как PDF
Версия для печати

В других проектах
Викисклад
Викиучебник
Элемент Викиданных

На других языках
Azərbaycanca
Deutsch
English
Español
Suomi
Հայերեն
Qaraqalpaqsha
Монгол
Српски / srpski
Ещё 69
Править ссылки

Объектно-ориентированное программирование

Материал из Википедии — свободной энциклопедии

Текущая версия страницы пока не проверялась опытными участниками и может значительно отличаться от версии, проверенной 15 марта 2022 года; проверки требует 78 правок.

[править | править код]

Объе́ктно-ориенти́рованное программи́рование (сокр. ООП) — методология или стиль программирования на основе описания типов/моделей предметной области и их взаимодействия, представленных порождением из прототипов или как экземпляры классов, которые образуют иерархию наследования[1].

Идеологически, ООП — подход к программированию как к моделированию информационных объектов, решающий на более высоком абстрактном уровне основную задачу структурного программирования — структурирование информации с точки зрения управляемости[2]. Это позволяет управлять самим процессом моделирования и реализовывать крупные программные проекты.

Управляемость для иерархических систем предполагает минимизацию избыточности данных (аналогичную нормализации) и их целостность, поэтому созданное удобно управляемым — будет и удобно пониматься. Таким образом, через тактическую задачу управляемости решается стратегическая задача — транслировать понимание задачи программистом в наиболее удобную для дальнейшего использования форму.

Методология ООП развивает и расширяет принципы объектного программирования[3].

Основные принципы структурирования в случае ООП связаны с различными аспектами базового понимания предметной задачи, которое требуется для оптимального управления соответствующей моделью[4]:

- абстрагирование выделяет существенные характеристики некоторого объекта, отличающие его от всех других видов объектов и, таким образом, чётко определяет его концептуальные границы с точки зрения наблюдателя
- инкапсуляция — это процесс отделения друг от друга элементов объекта, определяющих его структуру и поведение; инкапсуляция служит для того, чтобы изолировать контрактные обязательства абстракции от их реализации
- иерархия в виде наследования (генерализации) — такого отношения между классами (отношение родитель/потомок), когда один класс заимствует, а также расширяет и/или специализирует (уточняет) структуру и функциональный контракт одного или нескольких родительских классов
- модульность — это свойство системы, которая была разложена на внутренне связные, но слабо связанные между собой модули

Без следования любому из этих принципов модель не будет объектно-ориентированной.

То есть фактически речь идёт о прогрессирующей организации информации согласно первичным семантическим критериям: «важное/неважное», «ключевое/подробности», «родительское/дочернее», «единое/множественное». Прогрессирование, в частности, на последнем этапе даёт возможность перехода на следующий уровень детализации, что замыкает общий процесс.

Дополнительные свойства системы.

- типизация — это способ защититься от использования объектов одного класса вместо другого, или по крайней мере управлять таким использованием.
  - виртуальный полиморфизм - при динамическом связывании указатель (или ссылка) могут ссылаться на объект любого типа совместимого по иерархии наследования с типом указателя или ссылки соответственно
- параллелизм — это свойство, отличающее активные объекты от пассивных; параллелизм в объектно-ориентированном программировании, как и другие принципы, возник не на пустом месте, а явился результатом привнесения объектной идеи в теорию параллельных вычислений
- сохраняемость — это способность объекта существовать во времени, переживая породивший его процесс, и (или) в пространстве, перемещаясь из своего первоначального адресного пространства

Обычный человеческий язык в целом отражает идеологию ООП, начиная с инкапсуляции представления о предмете в виде его имени и заканчивая полиморфизмом использования слова в переносном смысле, что в итоге развивает[5] выражение представления через имя предмета до полноценного понятия-класса.

Парадигмы программирования

- Императивная (контрастирует с декларативной)
  - Процедурная
  - Структурная
  - Аспектно-ориентированная
  - **Объектно-ориентированная**
    - Агентно-ориентированная
    - Компонентно-ориентированная
    - Прототипно-ориентированная
  - Обобщённое программирование
- Декларативная (контрастирует с императивной)
  - Чистота языка
    - Чистота функции
  - Функциональная
    - В терминах рефал-машины
    - Аппликативная
    - Комбинаторная
    - Бесточечная
      - (чистая конкатенативная)
  - Логическая
    - Ограничениями
- Конкатенативная
- Векторная[англ.]
- Метапрограммирование
  - Языково-ориентированное
    - Предметно-ориентированное
    - Пользователями
  - Автоматизация процесса программирования
- Рефлексивность
  - Гомоиконичность
- Связанные темы
  - Программирование в крупном и мелком масштабе[англ.]
  - Модульность
  - Полиморфизм
  - Продолжения и CPS
  - Параллелизм
- Методы и алгоритмы
  - Автоматное
  - Потоков данных
  - Событийно-ориентированное
  - Реактивное
  - Сервис-ориентированное

Содержание [скрыть]

1 Основные понятия
2 Классификация подвидов ООП
3 История
4 Определение ООП и его основные концепции
4.1 Сложности определения
4.2 Концепции
5 Особенности реализации
6 Проектирование программ в целом
7 Различные ООП-методологии
7.1 Компонентное программирование
7.2 Прототипное программирование
7.3 Класс-ориентированное программирование
8 Производительность объектных программ
9 Критика ООП
10 Объектно ориентированные языки
11 См. также
12 Примечания
13 Литература
14 Ссылки

Основные понятия [править | править код]

Абстракция данных

Абстрагирование означает выделение значимой информации и исключение из рассмотрения незначимой. В ООП рассматривают лишь абстракцию данных (нередко называя её просто «абстракцией»), подразумевая набор наиболее значимых характеристик объекта, доступных остальной программе[источник не указан 130 дней].

Инкапсуляция[6]

Инкапсуляция — свойство системы, позволяющее объединить данные и методы, работающие с ними, в классе. Одни языки (например, С++, Java или Ruby) отождествляют инкапсуляцию с сокрытием, но другие (Smalltalk, Eiffel, OCaml) различают эти понятия.

Наследование[6]

Наследование — свойство системы, позволяющее описать новый класс на основе уже существующего с частично или полностью заимствованной функциональностью. Класс, от которого производится наследование, называется базовым, родительским или суперклассом. Новый класс — потомком, наследником, дочерним или производным классом.

Полиморфизм подтипов[6]

Полиморфизм подтипов (в ООП называемый просто «полиморфизмом») — свойство системы, позволяющее использовать объекты с одинаковым интерфейсом без информации о типе и внутренней структуре объекта. Другой вид полиморфизма — параметрический — в ООП называют обобщённым программированием.

Класс

Класс — универсальный, комплексный тип данных, состоящий из тематически единого набора «полей» (переменных более элементарных типов) и «методов» (функций для работы с этими полями), то есть он является моделью информационной сущности с внутренним и внешним интерфейсами для оперирования своим содержимым (значениями полей). В частности, в классах широко используются специальные блоки из одного или чаще двух спаренных методов, отвечающих за элементарные операции с определённым полем (интерфейс присваивания и считывания значения, геттер-сеттер), которые имитируют непосредственный доступ к полю. Эти блоки называются «свойствами» и почти совпадают по конкретному имени со своим полем (например, имя поля может начинаться со строчной, а имя свойства — с заглавной буквы). Другим проявлением интерфейсной природы класса является то, что при копировании соответствующей переменной через присваивание копируется только интерфейс, но не сами данные, то есть класс — ссылочный тип данных. Переменная-объект, относящаяся к заданному классом типу, называется экземпляром этого класса. При этом в некоторых исполняющих системах класс также может представляться некоторым объектом при выполнении программы посредством динамической идентификации типа данных. Обычно классы разрабатывают таким образом, чтобы обеспечить отвечающие природе объекта и решаемой задаче целостность данных объекта, а также удобный и простой интерфейс. В свою очередь, целостность предметной области объектов и их интерфейсов, а также удобство их проектирования, обеспечивается наследованием.

Объект

Сущность в адресном пространстве вычислительной системы, появляющаяся при создании экземпляра класса (например, после запуска результатов компиляции и связывания исходного кода на выполнение).

Классификация подвидов ООП [править | править код]

Лука Карделли[англ.] и Мартин Абади построили теоретическое обоснование ООП и классификацию на основе этого обоснования[7][8][9][10]. Они отмечают, что выделенные ими понятия и категории вместе встречаются далеко не во всех ОО-языках, большинство языков поддерживают лишь подмножества теории, а порой и своеобразные отклонения от неё.

Ключевые понятия:

- квантификация переменных типа (универсальная, экзистенциальная, ограниченная[англ.]);
- подтипизация[англ.] (англ. *subtyping* — отношения «супертип-подтип»);
- включение (англ. *subsumption* — см. принцип подстановки Барбары Лисков) — частный случай подтипизации — см. подтипизация на записях;
- объект;
- объектный тип (фактически, специальная форма «записи», в число полей которой входят функции первого класса);
- класс.

Класс описывает абстрактное поведение. Объектные типы строятся на основе класса посредством добавления различных приватных полей и методов. Объект (то есть значение объектного типа, традиционно называемое «экземпляром класса») порождается конструктором на основе начальных параметров.

Традиционно перечисляемые основные принципы ООП не апеллируют к данному теоретическому обоснованию, а являются устоявшимися в сообществе догмами (что приводит к сильному разбросу вариантов изложения в разных источниках). По большей части они относятся к языкам — потомкам Алгола и Симулы; в меньшей степени — к потомкам Smalltalk (в частности, часто упоминаемый принцип сокрытия в потомках Smalltalk семантически недоступен и идеологически считается несущественным). В значительно большей степени теоретически обоснованные концепции ООП поддерживают ОО-языки, развившиеся на поприще функционального программирования: OCaml, диалекты Haskell (O'Haskell, Mondrian), successor ML. Более того, основные идеи объектного моделирования в этом случае не требуют прямой поддержки со стороны языка, а могут быть довольно легко эмулированы[11].

Наиболее заметные отличия в проявлении показателей качества между языками разных видов:

- В мейнстримных языках декларируемые принципы нацелены на повышение изначально низкого для императивного программирования коэффициента повторного использования кода. В полиморфно типизированных применение концепций ООП, напротив, означает его очевидное снижение из-за перехода от параметрического полиморфизма к ad-hoc-полиморфизму[12]. В динамически типизированных языках (Smalltalk, Python, Ruby) эти принципы используются для логической организации программы, и их влияние на коэффициент повторного использования трудно спрогнозировать — он сильно зависит от дисциплины программиста. Например, в CLOS мультиметоды одновременно являются функциями первого класса, что позволяет рассматривать их одновременно и как связанно квантифицированные[англ.], и как обобщённые (истинно полиморфные).
- Традиционные ОО-языки используют номинативную типизацию[англ.], то есть допустимость соиспользования объектов разных классов только при условии явного указания родственных отношений между классами. Для полиморфно типизированных языков характерна структурная типизация, то есть согласование классов между собой тем же механизмом, что и согласование числа `5` с типом `int`. Динамически типизированные языки также занимают здесь промежуточную позицию.

Обобщённое обоснование динамической диспетчеризации[англ.] (включая множественную) в середине 1990-х годов построил Джузеппе Кастанья[13][14][15].

История [править | править код]

Этот раздел **нуждается в переработке**. **Пожалуйста, уточните проблему в разделе с помощью более узкого шаблона.**
Пожалуйста, улучшите статью в соответствии с правилами написания статей. *(24 апреля 2013)*

Объектно-ориентированное программирование возникло в результате развития идеологии процедурного программирования, где данные и подпрограммы (процедуры, функции) их обработки формально не связаны. Для дальнейшего развития объектно-ориентированного программирования часто большое значение имеют понятия события (так называемое событийно-ориентированное программирование) и компонента (компонентное программирование, КОП).

Взаимодействие объектов происходит посредством сообщений. Результатом дальнейшего развития ООП, по-видимому, будет агентно-ориентированное программирование, где *агенты* — независимые части кода на уровне выполнения. Взаимодействие агентов происходит посредством изменения *среды*, в которой они находятся.

Языковые конструкции, конструктивно не относящиеся непосредственно к объектам, но сопутствующие им для их безопасной (исключительные ситуации, проверки) и эффективной работы, инкапсулируются от них в аспекты (в аспектно-ориентированном программировании). Субъектно-ориентированное программирование расширяет понятие объекта посредством обеспечения более унифицированного и независимого взаимодействия объектов. Может являться переходной стадией между ООП и агентным программированием в части самостоятельного их взаимодействия.

Первым языком программирования, в котором были предложены основные понятия, впоследствии сложившиеся в парадигму, была Симула, но термин «объектная ориентированность» не использовался в контексте использования этого языка. В момент его появления в 1967 году в нём были предложены революционные идеи: объекты, классы, виртуальные методы и др., однако это всё не было воспринято современниками как нечто грандиозное. Фактически, Симула была «Алголом с классами», упрощающим выражение в процедурном программировании многих сложных концепций. Понятие класса в Симуле может быть полностью определено через композицию конструкций Алгола (то есть класс в Симуле — это нечто сложное, описываемое посредством примитивов).

Взгляд на программирование «под новым углом» (отличным от процедурного) предложили Алан Кэй и Дэн Ингаллс в языке Smalltalk. Здесь понятие класса стало основообразующей идеей для всех остальных конструкций языка (то есть класс в Смолтоке является примитивом, посредством которого описаны более сложные конструкции). Именно он стал первым широко распространённым объектно-ориентированным языком программирования.

В настоящее время количество прикладных языков программирования (список языков), реализующих объектно-ориентированную парадигму, является наибольшим по отношению к другим парадигмам. Наиболее распространённые в промышленности языки (С++, Delphi, C#, Java, Python и др.) воплощают объектную модель Симулы. Примерами языков, опирающихся на модель Смолтока, являются Objective-C, Python, Ruby.

Определение ООП и его основные концепции [править | править код]

В центре ООП находится понятие *объекта*. Объект — это сущность, которой можно посылать сообщения и которая может на них реагировать, используя свои данные. Объект — это экземпляр класса. Данные объекта скрыты от остальной программы. Инкапсуляция включает в себя сокрытие (Но им не является!).

Наличие инкапсуляции достаточно для объектности языка программирования, но ещё не означает его объектной ориентированности — для этого требуется наличие наследования.

Но даже наличие инкапсуляции и наследования не делает язык программирования в полной мере объектным с точки зрения ООП. Основные преимущества ООП проявляются только в том случае, когда в языке программирования реализован полиморфизм подтипов — возможность единообразно обрабатывать объекты с различной реализацией при условии наличия общего интерфейса.

Сложности определения [править | править код]

В этой статье имеется **избыток цитат либо слишком длинные цитаты**.
Излишние и чрезмерно большие цитаты следует обобщить и переписать своими словами. Возможно, эти цитаты будут более уместны в Викицитатнике или в Викитеке. *(16 марта 2014)*

ООП имеет уже более чем сорокалетнюю историю, но, несмотря на это, до сих пор не существует чёткого общепринятого определения данной технологии[16]. Основные принципы, заложенные в первые объектные языки и системы, подверглись существенному изменению (или искажению) и дополнению при многочисленных реализациях последующего времени. Кроме того, примерно с середины 1980-х годов термин «объектно-ориентированный» стал модным, в результате с ним произошло то же самое, что несколько раньше с термином «структурный» (ставшим модным после распространения технологии структурного программирования) — его стали искусственно «прикреплять» к любым новым разработкам, чтобы обеспечить им привлекательность. Бьёрн Страуструп в 1988 году писал, что обоснование «объектной ориентированности» чего-либо, в большинстве случаев, сводится к некорректному силлогизму: «X — это хорошо. Объектная ориентированность — это хорошо. *Следовательно*, X является объектно-ориентированным».

Тимоти Бадд пишет[17][18]:

> Роджер Кинг аргументированно настаивал, что его кот является объектно-ориентированным. Кроме прочих своих достоинств, кот демонстрирует характерное поведение, реагирует на сообщения, наделён унаследованными реакциями и управляет своим, вполне независимым, внутренним состоянием.

По мнению Алана Кэя, создателя языка Smalltalk, которого считают одним из «отцов-основателей» ООП, объектно ориентированный подход заключается в следующем наборе основных принципов (цитируется по вышеупомянутой книге Т. Бадда).

1. Всё является объектом.
2. Вычисления осуществляются путём взаимодействия (обмена данными) между объектами, при котором один объект требует, чтобы другой объект выполнил некоторое действие. Объекты взаимодействуют, посылая и получая сообщения. Сообщение — это запрос на выполнение действия, дополненный набором аргументов, которые могут понадобиться при выполнении действия.
3. Каждый объект имеет независимую память, которая состоит из других объектов.
4. Каждый объект является представителем класса, который выражает общие свойства объектов (таких, как целые числа или списки).
5. В классе задаётся поведение (функциональность) объекта. Тем самым все объекты, которые являются экземплярами одного класса, могут выполнять одни и те же действия.
6. Классы организованы в единую древовидную структуру с общим корнем, называемую иерархией наследования. Память и поведение, связанное с экземплярами определённого класса, автоматически доступны любому классу, расположенному ниже в иерархическом дереве.

Таким образом, программа представляет собой набор объектов, имеющих состояние и поведение. Объекты взаимодействуют посредством сообщений. Естественным образом выстраивается иерархия объектов: программа в целом — это объект, для выполнения своих функций она обращается к входящим в неё объектам, которые, в свою очередь, выполняют запрошенное путём обращения к другим объектам программы. Естественно, чтобы избежать бесконечной рекурсии в обращениях, на каком-то этапе объект трансформирует обращённое к нему сообщение в сообщения к стандартным системным объектам, предоставляемым языком и средой программирования.

Устойчивость и управляемость системы обеспечиваются за счёт чёткого разделения ответственности объектов (за каждое действие отвечает определённый объект), однозначного определения интерфейсов межобъектного взаимодействия и полной изолированности внутренней структуры объекта от внешней среды (инкапсуляции).

Определить ООП можно и многими другими способами.

Концепции [править | править код]

Появление в ООП отдельного понятия **класса** закономерно вытекает из желания иметь множество объектов со сходным поведением. Класс в ООП — это в чистом виде абстрактный тип данных, создаваемый программистом. С этой точки зрения объекты являются значениями данного абстрактного типа, а определение класса задаёт внутреннюю структуру значений и набор операций, которые над этими значениями могут быть выполнены. Желательность иерархии классов (а значит, наследования) вытекает из требований к повторному использованию кода — если несколько классов имеют сходное поведение, нет смысла дублировать их описание, лучше выделить общую часть в общий родительский класс, а в описании самих этих классов оставить только различающиеся элементы.

Необходимость совместного использования объектов разных классов, способных обрабатывать однотипные сообщения, требует поддержки **полиморфизма** — возможности записывать разные объекты в переменные одного и того же типа. В таких условиях объект, отправляя сообщение, может не знать в точности, к какому классу относится адресат, и одни и те же сообщения, отправленные переменным одного типа, содержащим объекты разных классов, вызовут различную реакцию.

Отдельного пояснения требует понятие обмена **сообщениями**. Первоначально (например, в том же Smalltalk) взаимодействие объектов представлялось как «настоящий» обмен сообщениями, то есть пересылка от одного объекта другому специального объекта-сообщения. Такая модель является чрезвычайно общей. Она прекрасно подходит, например, для описания параллельных вычислений с помощью *активных объектов*, каждый из которых имеет собственный поток исполнения и работает одновременно с прочими. Такие объекты могут вести себя как отдельные, абсолютно автономные вычислительные единицы. Посылка сообщений естественным образом решает вопрос обработки сообщений объектами, присвоенными полиморфным переменным — независимо от того, как объявляется переменная, сообщение обрабатывает код класса, к которому относится присвоенный переменной объект. Данный подход реализован в языках программирования Smalltalk, Ruby, Objective-C, Python.

Однако общность механизма обмена сообщениями имеет и другую сторону — «полноценная» передача сообщений требует дополнительных накладных расходов, что не всегда приемлемо. Поэтому во многих современных объектно-ориентированных языках программирования используется концепция **«отправка сообщения как вызов метода»** — объекты имеют доступные извне методы, вызовами которых и обеспечивается взаимодействие объектов. Данный подход реализован в огромном количестве языков программирования, в том числе C++, Object Pascal, Java, Oberon-2. Однако, это приводит к тому, что сообщения уже не являются самостоятельными объектами, и, как следствие, не имеют атрибутов, что сужает возможности программирования. Некоторые языки используют гибридное представление, демонстрируя преимущества одновременно обоих подходов — например, CLOS, Python.

Концепция **виртуальных методов**, поддерживаемая этими и другими современными языками, появилась как средство обеспечить выполнение нужных методов при использовании полиморфных переменных, то есть, по сути, как попытка расширить возможности вызова методов для реализации части функциональности, обеспечиваемой механизмом обработки сообщений.

Особенности реализации [править | править код]

Как уже говорилось выше, в современных объектно-ориентированных языках программирования каждый объект является значением, относящимся к определённому классу. Класс представляет собой объявленный программистом составной тип данных, имеющий в составе:

Поля данных

Параметры объекта (конечно, не все, а только необходимые в программе), задающие его состояние (свойства объекта предметной области). Иногда поля данных объекта называют свойствами объекта, из-за чего возможна путаница. Фактически поля представляют собой значения (переменные, константы), объявленные как принадлежащие классу.

Методы

Процедуры и функции, связанные с классом. Они определяют действия, которые можно выполнять над объектом такого типа, и которые сам объект может выполнять.

Классы могут наследоваться друг от друга. Класс-потомок получает все поля и методы класса-родителя, но может дополнять их собственными либо переопределять уже имеющиеся. Большинство языков программирования поддерживает только единичное наследование (класс может иметь только один класс-родитель), лишь в некоторых допускается множественное наследование — порождение класса от двух или более классов-родителей. Множественное наследование создаёт целый ряд проблем, как логических, так и чисто реализационных, поэтому в полном объёме его поддержка не распространена. Вместо этого в 1990-е годы появилось и стало активно вводиться в объектно-ориентированные языки понятие интерфейса. Интерфейс — это класс без полей и без реализации, включающий только заголовки методов. Если некий класс наследует (или, как говорят, реализует) интерфейс, он должен реализовать все входящие в него методы. Использование интерфейсов предоставляет относительно дешёвую альтернативу множественному наследованию.

Взаимодействие объектов в абсолютном большинстве случаев обеспечивается вызовом ими методов друг друга.

Инкапсуляция обеспечивается следующими средствами:

Контроль доступа

Поскольку методы класса могут быть как чисто внутренними, обеспечивающими логику функционирования объекта, так и внешними, с помощью которых взаимодействуют объекты, необходимо обеспечить скрытость первых при доступности извне вторых. Для этого в языки вводятся специальные синтаксические конструкции, явно задающие область видимости каждого члена класса. Традиционно это модификаторы public, protected и private, обозначающие, соответственно, открытые члены класса, члены класса, доступные внутри класса и из классов-потомков, и скрытые, доступные только внутри класса. Конкретная номенклатура модификаторов и их точный смысл различаются в разных языках.

Методы доступа

Поля класса в общем случае не должны быть доступны извне, поскольку такой доступ позволил бы произвольным образом менять внутреннее состояние объектов. Поэтому поля обычно объявляются скрытыми (либо язык в принципе не позволяет обращаться к полям класса извне), а для доступа к находящимся в полях данным используются специальные методы, называемые методами доступа. Такие методы либо возвращают значение того или иного поля, либо производят запись в это поле нового значения. При записи метод доступа может проконтролировать допустимость записываемого значения и, при необходимости, произвести другие манипуляции с данными объекта, чтобы они остались корректными (внутренне согласованными). Методы доступа называют ещё аксессорами (от англ. *access* — доступ), а по отдельности — геттерами (англ. *get* — получить) и сеттерами (англ. *set* — установить)[19].

Свойства объекта

Псевдополя, доступные для чтения и/или записи. Свойства внешне выглядят как поля и используются аналогично доступным полям (с некоторыми исключениями), однако фактически при обращении к ним происходит вызов методов доступа. Таким образом, свойства можно рассматривать как «умные» поля данных, сопровождающие доступ к внутренним данным объекта какими-либо дополнительными действиями (например, когда изменение координаты объекта сопровождается его перерисовкой на новом месте). Свойства, по сути, не более чем синтаксический сахар, поскольку никаких новых возможностей они не добавляют, а лишь скрывают вызов методов доступа. Конкретная языковая реализация свойств может быть разной. Например, в C# объявление свойства непосредственно содержит код методов доступа, который вызывается только при работе со свойствами, то есть не требует отдельных методов доступа, доступных для непосредственного вызова. В Delphi объявление свойства содержит лишь имена методов доступа, которые должны вызываться при обращении к полю. Сами методы доступа представляют собой обычные методы с некоторыми дополнительными требованиями к сигнатуре.

Полиморфизм реализуется путём введения в язык правил, согласно которым переменной типа «класс» может быть присвоен объект любого класса-потомка её класса.

Проектирование программ в целом [править | править код]

ООП ориентировано на разработку крупных программных комплексов, разрабатываемых командой программистов (возможно, достаточно большой). Проектирование системы в целом, создание отдельных компонентов и их объединение в конечный продукт при этом часто выполняется разными людьми, и нет ни одного специалиста, который знал бы о проекте всё.

Объектно-ориентированное проектирование ориентируется на описание структуры проектируемой системы (приоритетно по отношению к описанию её поведения, в отличие от функционального программирования), то есть, фактически, в ответе на два основных вопроса:

- **Из каких частей состоит система;**
- **В чём состоит ответственность каждой из её частей.**

Выделение частей производится таким образом, чтобы каждая имела минимальный по объёму и точно определённый набор выполняемых функций (обязанностей), и при этом взаимодействовала с другими частями как можно меньше.

Дальнейшее уточнение приводит к выделению более мелких фрагментов описания. По мере детализации описания и определения ответственности выявляются данные, которые необходимо хранить, наличие близких по поведению агентов, которые становятся кандидатами на реализацию в виде классов с общими предками. После выделения компонентов и определения интерфейсов между ними реализация каждого компонента может проводиться практически независимо от остальных (разумеется, при соблюдении соответствующей технологической дисциплины).

Большое значение имеет правильное построение иерархии классов. Одна из известных проблем больших систем, построенных по ООП-технологии — так называемая *проблема хрупкости базового класса*. Она состоит в том, что на поздних этапах разработки, когда иерархия классов построена и на её основе разработано большое количество кода, оказывается трудно или даже невозможно внести какие-либо изменения в код базовых классов иерархии (от которых порождены все или многие работающие в системе классы). Даже если вносимые изменения не затронут интерфейс базового класса, изменение его поведения может непредсказуемым образом отразиться на классах-потомках. В случае крупной системы разработчик базового класса просто не в состоянии предугадать последствия изменений, он даже не знает о том, как именно базовый класс используется и от каких особенностей его поведения зависит корректность работы классов-потомков.

Различные ООП-методологии [править | править код]

Компонентное программирование — следующий этап развития ООП; прототип- и класс-ориентированное программирование — разные подходы к созданию программы, которые могут комбинироваться, имеющие свои преимущества и недостатки.

Компонентное программирование [править | править код]

Основная статья: ***Компонентно-ориентированное программирование***

Компонентно-ориентированное программирование — это своеобразная «надстройка» над ООП, набор правил и ограничений, направленных на построение крупных развивающихся программных систем с большим временем жизни. Программная система в этой методологии представляет собой набор компонентов с хорошо определёнными интерфейсами. Изменения в существующую систему вносятся путём создания новых компонентов в дополнение или в качестве замены ранее существующих. При создании новых компонентов на основе ранее созданных запрещено использование наследования реализации — новый компонент может наследовать лишь интерфейсы базового. Таким образом, компонентное программирование обходит проблему хрупкости базового класса.

Прототипное программирование [править | править код]

Прототипное программирование, сохранив часть черт ООП, отказалось от базовых понятий — класса и наследования.

- Прототип — это объект-образец, по образу и подобию которого создаются другие объекты. Объекты-копии могут сохранять связь с родительским объектом, автоматически наследуя изменения в прототипе; эта особенность определяется в рамках конкретного языка.
- Вместо механизма описания классов и порождения экземпляров, язык предоставляет механизм создания объекта (путём задания набора полей и методов, которые объект должен иметь) и механизм клонирования объектов.
- Каждый вновь созданный объект является «экземпляром без класса». Каждый объект может стать *прототипом* — быть использован для создания нового объекта с помощью операции *клонирования*. После клонирования объект может быть изменён, в частности, дополнен новыми полями и методами.
- Клонированный объект либо становится полной копией прототипа, хранящей все значения его полей и дублирующей его методы, либо сохраняет ссылку на прототип, не включая в себя клонированных полей и методов до тех пор, пока они не будут изменены. В последнем случае среда исполнения обеспечивает механизм *делегирования* — если при обращении к объекту он сам не содержит нужного метода или поля данных, вызов передаётся прототипу, от него, при необходимости — дальше по цепочке.

Класс-ориентированное программирование [править | править код]

В разделе **не хватает ссылок на источники** (см. рекомендации по поиску).
Информация должна быть проверяема, иначе она может быть удалена. Вы можете отредактировать статью, добавив ссылки на авторитетные источники в виде сносок. *(31 декабря 2012)*

Класс-ориентированное программирование — это программирование, сфокусированное на данных, причём данные и поведение неразрывно связаны между собой. Вместе данные и поведение представляют собой класс. Соответственно в языках, основанных на понятии «класс», все объекты разделены на два основных типа — классы и экземпляры. Класс определяет структуру и функциональность (поведение), одинаковую для всех экземпляров данного класса. Экземпляр является носителем данных — то есть обладает состоянием, меняющимся в соответствии с поведением, заданным классом. В класс-ориентированных языках новый экземпляр создаётся через вызов конструктора класса (возможно, с набором параметров). Получившийся экземпляр имеет структуру и поведение, жёстко заданные его классом.

Производительность объектных программ [править | править код]

Гради Буч указывает[20] на следующие причины, приводящие к снижению производительности программ из-за использования объектно-ориентированных средств:

Динамическое связывание методов

Обеспечение полиморфного поведения объектов приводит к необходимости связывать методы, вызываемые программой (то есть определять, какой конкретно метод будет вызываться) не на этапе компиляции, а в процессе исполнения программы, на что тратится дополнительное время. При этом реально динамическое связывание требуется не более чем для 20 % вызовов, но некоторые ООП-языки используют его постоянно.

Значительная глубина абстракции

ООП-разработка часто приводит к созданию «многослойных» приложений, где выполнение объектом требуемого действия сводится к множеству обращений к объектам более низкого уровня. В таком приложении происходит очень много вызовов методов и возвратов из методов, что, естественно, сказывается на производительности.

Наследование «размывает» код

Код, относящийся к «конечным» классам иерархии наследования, которые обычно и используются программой непосредственно, находится не только в самих этих классах, но и в их классах-предках. Относящиеся к одному классу методы фактически описываются в разных классах. Это приводит к двум неприятным моментам:

- **Снижается скорость трансляции, так как компоновщику приходится подгружать описания всех классов иерархии.**
- **Снижается производительность программы в системе со страничной памятью — поскольку методы одного класса физически находятся в разных местах кода, далеко друг от друга, при работе фрагментов программы, активно обращающихся к унаследованным методам, система вынуждена производить частые переключения страниц.**

Инкапсуляция снижает скорость доступа к данным

Запрет на прямой доступ к полям класса извне приводит к необходимости создания и использования методов доступа. И написание, и компиляция, и исполнение методов доступа сопряжены с дополнительными расходами.

Динамическое создание и уничтожение объектов

Динамически создаваемые объекты, как правило, размещаются в куче, что менее эффективно, чем размещение их на стеке и, тем более, статическое выделение памяти под них на этапе компиляции.

Несмотря на отмеченные недостатки, Буч утверждает, что выгоды от использования ООП более весомы. Кроме того, повышение производительности за счёт лучшей организации ООП-кода, по его словам, в некоторых случаях компенсирует дополнительные накладные расходы на организацию функционирования программы. Можно также заметить, что многие эффекты снижения производительности могут сглаживаться или даже полностью устраняться за счёт качественной оптимизации кода компилятором. Например, упомянутое выше снижение скорости доступа к полям класса из-за использования методов доступа устраняется, если компилятор вместо вызова метода доступа использует инлайн-подстановку (современные компиляторы делают это вполне уверенно).

Критика ООП [править | править код]

В этой статье имеется **избыток цитат либо слишком длинные цитаты**.
Излишние и чрезмерно большие цитаты следует обобщить и переписать своими словами. Возможно, эти цитаты будут более уместны в Викицитатнике или в Викитеке. *(16 марта 2014)*

Несмотря на отдельные критические замечания в адрес ООП, в настоящее время именно эта парадигма используется в подавляющем большинстве промышленных проектов. Однако нельзя считать, что ООП является наилучшей из методик программирования во всех случаях.

Критические высказывания в адрес ООП:

- Было показано отсутствие значимой разницы в продуктивности разработки программного обеспечения между ООП и процедурным подходом[21].
- Кристофер Дэйт указывает на невозможность сравнения ООП и других технологий во многом из-за отсутствия строгого и общепризнанного определения ООП[22].
- Александр Степанов в одном из своих интервью указывал, что ООП «методологически неправильно» и что «…ООП практически такая же мистификация, как и искусственный интеллект…»[23].
- Фредерик Брукс указывает, что наиболее сложной частью создания программного обеспечения является «…спецификация, дизайн и тестирование концептуальных конструкций, а отнюдь не работа по выражению этих концептуальных конструкций…». ООП (наряду с такими технологиями как искусственный интеллект, верификация программ, автоматическое программирование, графическое программирование, экспертные системы и др.), по его мнению, не является «серебряной пулей», которая могла бы на порядок снизить сложность разработки программных систем. Согласно Бруксу, «…ООП позволяет сократить только привнесённую сложность в выражение дизайна. Дизайн остаётся сложным по своей природе…»[24].
- Эдсгер Дейкстра указывал: «…то, о чём общество в большинстве случаев просит — это эликсир от всех болезней. Естественно, "эликсир" имеет очень впечатляющие названия, иначе будет очень трудно что-то продать: "Структурный анализ и Дизайн", "Программная инженерия", "Модели зрелости", "Управляющие информационные системы" (Management Information Systems), "Интегрированные среды поддержки проектов", "Объектная ориентированность", "Реинжиниринг бизнес-процессов"…»[25].
- Никлаус Вирт считает, что ООП — не более чем тривиальная надстройка над структурным программированием[источник не указан 2316 дней], и преувеличение её значимости, выражающееся, в том числе, во включении в языки программирования всё новых модных «объектно-ориентированных» средств, вредит качеству разрабатываемого программного обеспечения.
- Патрик Киллелиа в своей книге «Тюнинг веб-сервера» писал: «…ООП предоставляет вам множество способов замедлить работу ваших программ…».
- Известная обзорная статья проблем современного ООП-программирования перечисляет некоторые типичные проблемы ООП[26][неавторитетный источник].
- В программистском фольклоре получила широкое распространение критика объектно-ориентированного подхода в сравнении с функциональным подходом с использованием метафоры «*Действия в Королевстве существительных*» из эссе Стива Йегге[27].

Если попытаться классифицировать критические высказывания в адрес ООП, можно выделить несколько аспектов критики данного подхода к программированию.

Критика рекламы ООП

Критикуется явно высказываемое или подразумеваемое в работах некоторых пропагандистов ООП, а также в рекламных материалах «объектно-ориентированных» средств разработки представление об объектном программировании как о некоем всемогущем подходе, который магическим образом устраняет сложность программирования. Как замечали многие, в том числе упомянутые выше Брукс и Дейкстра, «серебряной пули не существует» — независимо от того, какой парадигмы программирования придерживается разработчик, создание нетривиальной сложной программной системы всегда сопряжено со значительными затратами интеллектуальных ресурсов и времени. Из наиболее квалифицированных специалистов в области ООП никто, как правило, не отрицает справедливость критики этого типа.

Оспаривание эффективности разработки методами ООП

Критики оспаривают тезис о том, что разработка объектно-ориентированных программ требует меньше ресурсов или приводит к созданию более качественного ПО. Проводится сравнение затрат на разработку разными методами, на основании которого делается вывод об отсутствии у ООП преимуществ в данном направлении. Учитывая крайнюю сложность объективного сравнения различных разработок, подобные сопоставления, как минимум, спорны. С другой стороны, получается, что ровно так же спорны и утверждения об эффективности ООП.

Производительность объектно ориентированных программ

Указывается на то, что целый ряд «врождённых особенностей» ООП-технологий делает построенные на её основе программы технически менее эффективными, по сравнению с аналогичными необъектными программами. Не отрицая действительно имеющихся дополнительных накладных расходов на организацию работы ООП-программ (см. раздел «Производительность» выше), однако, значение снижения производительности часто преувеличивается критиками. В современных условиях, когда технические возможности компьютеров чрезвычайно велики и постоянно растут, для большинства прикладных программ техническая эффективность оказывается менее существенна, чем функциональность, скорость разработки и сопровождаемость. Лишь для некоторого, очень ограниченного класса программ (ПО встроенных систем, драйверы устройств, низкоуровневая часть системного ПО, научное ПО) производительность остаётся критическим фактором.

Критика отдельных технологических решений в ООП-языках и библиотеках

Эта критика многочисленна, но затрагивает она не ООП как таковое, а приемлемость и применимость в конкретных случаях тех или иных реализаций её механизмов. Одним из излюбленных объектов критики является язык C++, входящий в число наиболее распространённых промышленных ООП-языков.

Объектно ориентированные языки [править | править код]

Основная статья: ***Объектно-ориентированный язык программирования***

Многие современные языки специально созданы для облегчения объектно-ориентированного программирования. Однако можно применять техники ООП и для не объектно-ориентированного языка и наоборот, применение объектно-ориентированного языка вовсе не означает, что код автоматически становится объектно-ориентированным.

Как правило, объектно-ориентированный язык (ООЯ) содержит следующий набор элементов:

- Объявление классов с полями (данными — членами класса) и методами (функциями — членами класса).
- Механизм расширения класса (наследования) — порождение нового класса от существующего с автоматическим включением всех особенностей реализации класса-предка в состав класса-потомка. Большинство ООЯ поддерживают только единичное наследование.
- Полиморфные переменные и параметры функций (методов), позволяющие присваивать одной и той же переменной экземпляры различных классов.
- Полиморфное поведение экземпляров классов за счёт использования виртуальных методов. В некоторых ООЯ все методы классов являются виртуальными.

Некоторые языки добавляют к указанному минимальному набору те или иные дополнительные средства. В их числе:

- Конструкторы, деструкторы, финализаторы;
- Свойства (аксессоры);
- Индексаторы;
- Средства управления видимостью компонентов классов (модификаторы доступа или интерфейсы, такие как public, private, protected, feature и др.).

Одни языки отвечают принципам ООП в полной мере — в них все основные элементы являются объектами, имеющими состояние и связанные методы. Примеры подобных языков — Smalltalk, Eiffel. Существуют гибридные языки, совмещающие объектную подсистему в целостном виде с подсистемами других парадигм как «два и более языка в одном», позволяющие совмещать в одной программе объектные модели с иными, и размывающие грань между объектно-ориентированной и другими парадигмами за счёт нестандартных возможностей, балансирующих между ООП и другими парадигмами (таких как множественная диспетчеризация, параметрические классы, возможность манипулировать методами классов как самостоятельными объектами, и др.). Примеры таких языков: CLOS, Dylan, OCaml, Python, Ruby, Objective-C. Однако, наиболее распространены языки, включающие средства эмуляции объектной модели поверх более традиционной императивной семантики. Алан Кэй назвал такие языки «склеиванием возможностей» (англ. *agglutination of features*) в противовес «чистоте стиля» (англ. *crystalization of style*) языков, воплощающих некую парадигму непосредственно[28]. Примеры таких языков — Симула, C++, Visual Basic, Delphi, Модула, Модула-2, Java, C#, PHP.

См. также [править | править код]

- ORM — технология связывания ОО-программ с базами данных
- Доменный объект
- UML
- Сравнение языков программирования

Примечания [править | править код]

↑ Показать компактно

1. ↑ Буч, 1998.
2. ↑ Edsger W. Dijkstra Программирование как вид человеческой деятельности. 1979 (EWD117). Дата обращения: 26 февраля 2019. Архивировано 7 августа 2018 года.
3. ↑ В. В. Мухортов, В. Ю. Рылов, 2002, с. 22—24: «Раздел 1.2.2 Объектно-ориентированные языки».
4. ↑ В. В. Мухортов, В. Ю. Рылов, 2002, с. 26—43: «Глава 2. Составные части объектного подхода».
5. ↑ Л. В. Успенский. «Слово о словах». — 5-е изд. — Л.: Детская литература (Ленинградское отделение), 1971. Дата обращения: 26 февраля 2019. Архивировано 24 февраля 2019 года.
6. ↑ 1 2 3 Пирс, 2012.
7. ↑ Abadi, Cardelli, 1994.
8. ↑ Cardelli, 1991.
9. ↑ Cardelli, 1985.
10. ↑ Пирс, 2012.
11. ↑ Object-Oriented Programming in Standard ML. Дата обращения: 3 февраля 2016. Архивировано 14 января 2016 года.
12. ↑ Cardelli, 1985, 1.3. Kinds of Polymorphism, с. 6.
13. ↑ Castagna, Ghelli, Longo. A calculus for overloaded functions with subtyping. // Information and Computation. — Academic press, 1995. — Т. 117, вып. 1. — С. 115–135. — doi:10.1006/inco.1995.1033. Архивировано 18 ноября 2018 года.
14. ↑ Castagna. Object-Oriented Programming: A Unified Foundation. — Birkhäuser, 1996. — С. 384. — ISBN 978-0-8176-3905-1.
15. ↑ Пирс, 2012.
16. ↑ Nobody Agrees On What OO Is. Дата обращения: 14 июля 2012. Архивировано 14 июля 2012 года.
17. ↑ *Тимоти Бадд.* Объектно-ориентированное программирование в действии = An Introduction to Object-Oriented Programming. — СПб.: «Питер», 1997. — 464 с. — (В действии). — 6000 экз. — ISBN 5-88782-270-8.
18. ↑ Roger King, My cat is object-oriented
19. ↑ Следует, однако, заметить, что использование аксессоров подрывает инкапсуляцию, суть которой состоит не в сокрытии всех членов объекта, а в том, что внутреннее состояние объекта используется лишь для работы, совершаемой им непосредственно. В ООП программист работает с понятиями, а не с данными.
20. ↑ Буч, 1998, с. 276—278.
21. ↑ *Thomas E. Potok, Mladen Vouk, Andy Rindos.* Productivity Analysis of Object-Oriented Software Developed in a Commercial Environment (англ.) // Software — Practice and Experience. — 1999. — Vol. 29, no. 10. — P. 833—847. Архивировано 6 января 2007 года.
22. ↑ C. J. Date, Introduction to Database Systems, 6th-ed., Page 650
23. ↑ STLport: An Interview with A. Stepanov. Дата обращения: 27 марта 2007. Архивировано 4 марта 2009 года.
24. ↑ «No Silver Bullet. Essence and Accidents of Software Engineering» (Computer Magazine; April 1987)
25. ↑ EWD 1175: The strengths of the academic enterprise. Дата обращения: 6 августа 2007. Архивировано 31 октября 2007 года.
26. ↑ Почему объектно-ориентированное программирование провалилось. Дата обращения: 22 сентября 2010. Архивировано 23 сентября 2010 года.
27. ↑ *Steve Yegge.* Execution in the Kingdom of Nouns. Дата обращения: 9 сентября 2015. Архивировано 5 сентября 2015 года. (частичный перевод Архивная копия от 2 мая 2015 на Wayback Machine (рус.))
28. ↑ *Алан Кэй.* The Early History of Smalltalk. — Apple Computer, ACM SIGPLAN Notices, vol.28, №3, March 1993. Архивировано 14 сентября 2012 года.

Литература [править | править код]

- *Бадд Т.* Объектно-ориентированное программирование в действии = An Introduction to Object-Oriented Programming. — СПб.: Питер, 1997. — ISBN 5-88782-270-8.
- *Буч Г.* Объектно-ориентированный анализ и проектирование с примерами приложений на C++ = Object-Oriented Analysis and Design with Applications / Пер. И.Романовский, Ф.Андреев. — 2-е изд. — М., СПб.: «Бином», «Невский диалект», 1998. — 560 с. — 6000 экз. — ISBN 5-7989-0067-3.
- *Буч Г., Роберт А. Максимчук, Майкл У. Энгл, Бобби Дж. Янг, Джим Коналлен, Келли А. Хьюстон.* Объектно-ориентированный анализ и проектирование с примерами приложений. — 3-е издание. — «Вильямс», 2010. — 720 с. — ISBN 978-5-8459-1401-9. — ISBN 0-201-89551-X.
- *Иан Грэхем.* Объектно-ориентированные методы. Принципы и практика = Object-Oriented Methods: Principles & Practice. — 3-е изд. — М.: «Вильямс», 2004. — С. 880. — ISBN 0-201-61913-X.
- *В. В. Мухортов, В. Ю. Рылов.* Объектно-ориентированное программирование, анализ и дизайн. Методическое пособие. — 2002. — 108 с. — ISBN - УДК 519.682.
- *Лука Карделли*[англ.]. Typeful programming (англ.) // IFIP State-of-the-Art Reports. — New York: Springer-Verlag, 1991. — Вып. Formal Description of Programming Concepts.
- *Лука Карделли*[англ.], *Peter Wegner.* On Understanding Types, Data Abstraction, and Polymorphism // ACM Computing Surveys. — New York, USA: ACM, 1985. — Т. 17, вып. 4. — С. 471—523. — ISSN 0360-0300. — doi:10.1145/6041.6042.
- *Martin Abadi, Лука Карделли*[англ.]. A Semantics of Object Types (англ.). — LICS[англ.], 1994.
- *Мартин Р.* Часть II. Парадигмы программирования. // Чистая архитектура = Clean Architecture. — СПб.: Питер, 2018. — ISBN 978-5-4461-0772-8.
- *Майер Б.* Объектно-ориентированное конструирование программных систем. — М.: Русская редакция, 2005. — ISBN 5-7502-0255-0.
- *Benjamin Pierce.* Types and Programming Languages (англ.). — MIT Press, 2002. — ISBN 0-262-16209-1.
  - Перевод на русский язык: *Бенджамин Пирс.* Типы в языках программирования (неопр.). — Добросвет, 2012. — 680 с. — ISBN 978-5-7913-0082-9.
- *Антони Синтес.* Освой самостоятельно объектно-ориентированное программирование за 21 день = Sams Teach Yourself Object-Oriented Programming in 21 Days. — М.: «Вильямс», 2002. — С. 672. — ISBN 0-672-32109-2.
- *Усов Т. М.* Введение в объектно-ориентированное программирование с примерами на C#. 2020 - ISBN 978-5-600-02430-4. Архивная копия от 23 мая 2020 на Wayback Machine
- Design Patterns
- *Matt Weisfeld.* The Object-Oriented Thought Process. — Fourth Edition. — Addison-Wesley Professional, 2013. — 336 с. — ISBN 978-0-321-86127-6.

Ссылки [править | править код]

Обзоры

- Обзор Объектно Ориентированного Программирования Архивная копия от 5 февраля 2017 на Wayback Machine (рус.)
- Основы объектно ориентированного программирования Архивная копия от 2 февраля 2017 на Wayback Machine (рус.) Интуит.ру Бертран Мейер
- С.Бобровский «История ООП» Архивная копия от 22 февраля 2008 на Wayback Machine, PC Week/RE № 28 от 05.08.2003 г., стр. 10, и № 29 от 12.08.2003 г., стр. 10.

Критика ООП

- Ричард П. Габриэл «Объектная парадигма провалилась» Архивная копия от 4 января 2009 на Wayback Machine
- Гай Л. Стил «Объектная парадигма не провалилась» Архивная копия от 3 января 2009 на Wayback Machine

В родственных проектах
Книги в Викиучебнике
Медиафайлы на Викискладе

Словари и энциклопедии: Большая советская · Большая норвежская · Большая российская (научно-образовательный портал) · Britannica (онлайн) · De Agostini
В библиографических каталогах [показать]
Разработка программного обеспечения [показать]
Парадигмы программирования

Категории: Объектно-ориентированное программирование | Парадигмы программирования

Эта страница в последний раз была отредактирована 12 февраля 2025 в 03:45.
Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия. Подробнее см. Условия использования.
Wikipedia® — зарегистрированный товарный знак некоммерческой организации «Фонд Викимедиа» (Wikimedia Foundation, Inc.)